

F1R3COMB-MAT

The rho combinators as sparse linear algebra:
a Rust implementation of the matrix representation,
the bulk-synchronous machine, and the transfer operator
Specification v0.1 — companion to F1R3COMB v0.2

F1R3FLY.io

22 September 2026

Abstract

Rho combinators as sparse linear algebra [1] shows that a combinator state is an occupation vector, that the one-step relation is a sum of ladder-operator monomials, and — the practical finding — that the matrix worth multiplying is not the transition matrix of the reduction graph but the incidence of atom occurrences against names, which is term-sized. Redex finding is a join on that incidence, conflict detection is a second sparse product, and a machine step fires a maximal independent set of redexes, so machine steps count causal depth. A Python reference (`rhocombmat.py`, `experiments.py`) validates the join against naive enumeration and measures width on four term families. F1R3COMB v0.2 [2] specifies the compiler from kernel rho to the combinators and an interleaving machine built on CAMPF1R3. Nothing takes a combinator term to its matrix representation in Rust, and nothing runs it there.

This document specifies that arrow and what sits on it: three crates in the `campfir3` workspace that intern a F1R3COMB term into a fixed-width, integer-indexed representation; find redexes by joins derived from one shared rule table; select a maximal independent set by a bit-exactly specified priority function; fire it; and, on the destructor-free sublattice, materialise the transfer operator over reachable markings and close it. The design keeps every invariant F1R3COMB v0.2 established — presentation A, the `m/q` sort asymmetry, canonical encodings, no external dependencies, bit-identical results across core counts and WebAssembly — and reconciles the one place the two documents appear to disagree, the payload of `q` (Decision 2.2).

Checking the note against its own reference turned up three errors that the implementation cannot inherit, because it asserts the invariants in question. The atom-count lemma of draft 3 [3, Lem. 9.2] is false for `bl`, `br` and `s`, so the length bound of [3, Prop. 9.9] and the nilpotency index of [1, Thm. 6.1(ii)] are wrong, though both conclusions survive under a weighted grading (Erratum 9.1). The message-count invariant of [1, Thm. 6.1(iii)] is broken by the `quote` rule and holds for messages plus stores (Erratum 9.2). And the reference enumerates symmetric constructor matches twice, which is harmless to the step and wrong for the mass-action coefficient (Erratum 6.3). Section 16 lists these with proposed amendments. Key words MUST, MUST NOT, SHOULD, MAY are used in the sense of RFC 2119.

Contents

1	Purpose and scope	3
1.1	The gap	3
1.2	What exists today	4
1.3	Nomenclature	4
1.4	Goals	5
1.5	Non-goals	5

2	Position relative to F1R3COMB	5
2.1	Two machines, one term type	5
2.2	Decisions inherited and decisions taken	6
3	The representation	7
3.1	Names and the intern table	7
3.2	State: shape tables	8
3.3	Import and export	8
3.4	The canonical export format	8
4	The rule table, as F1R3COMB-MAT reads it	9
4.1	The table	9
4.2	Rule classes	9
4.3	Join plans	10
5	Sparse algebra	10
6	Redex finding	10
6.1	The redex	10
6.2	Three finders	11
6.3	Contention	11
7	Conflict and selection	12
7.1	The conflict structure	12
7.2	The priority function	12
7.3	The step	12
7.4	Resolvers	13
8	Firing	13
9	The grading, and what it bounds	14
10	The operator layer	14
10.1	Species and markings	15
10.2	Successors: one column of T , lazily	15
10.3	Exploration and the materialised operator	15
11	Instrumentation and the width gate	16
12	Architecture	17
12.1	The crates	17
12.2	One step	17
12.3	Backends	17
12.4	The device seam	18
13	Correctness obligations and testing	18
14	Errors and diagnostics	19
15	Performance, portability, determinism	19
16	What the implementation asks of the papers	20
17	Open questions and deferred work	20

1 Purpose and scope

1.1 The gap

[1] is a design analysis with a correctness reference. Its six claims (C1)–(C6) are about a representation and a step discipline; its reference implementation is explicitly “a correctness reference, not a performance artefact”; and its development plan (§11, phases P0–P6) puts the Rust port at P3, after a width measurement (P2) that needs the compiler. The compiler now has a specification [2]. This document is P0, P3 and P6 of [1], specified against F1R3COMB’s term type, and it supplies the instrument P2 needs.

Two properties the Python reference is entitled to leave loose are properties this implementation is not. The reference assigns name indices and occurrence identities by insertion into Python dictionaries, and chooses the independent set by a shuffled order; its runs are reproducible only by seed and only in one process. F1R3COMB v0.2 requires the compiled term, its hash, the event stream and the final scene to be bit-identical across runs, hosts, core counts and WebAssembly ([2, Req. 12.4]). A second machine over the same terms that cannot meet that requirement cannot be differentially tested against the first, and cannot be ported to a device with the exit criterion [1, P4] sets — identical step traces under a fixed seed.

1.2 What exists today

Artefact	What it gives this project
<code>rho-combinators/rhocomb-matrix</code>	The state space (§4), the transfer operator and its three rule classes (§5), the lattice-as-finiteness theorem (Thm. 6.1), the incidence matrix and the join table (§7), the step and its soundness (§8), reflection as an intern table (§9), measurements (§10), the plan and obligations (§11, §13)
<code>rho-combinators/rhocombmat.py</code>	The reference machine: intern table, occurrence store, incidence, joins, conflict product, greedy independent set, firing
<code>rho-combinators/experiments.py, results.txt</code>	Four term families E1–E4 and a 400-state validation; every number in [1, §10]
<code>rho-combinators/f1r3comb-spec-v02</code>	The target term type (<code>f1r3comb-term</code>), presentation A, the encoding, the tag table, the rule table requirement, the CAMPF1R3 machine, the diagnostics and portability envelope this document inherits
<code>rho-combinators/rho-combinators-draft3</code>	The calculus (Def. 3.3), the lattice invariants (§9), the shortcuts (Rem. 6.7)
<code>rho-combinators/rhocomb-logic</code>	Components as a complete invariant (Def. 2.9), drop elimination (Lem. 2.8), injective quotation (Prop. 2.10), occupancy (Thm. 6.5); the eventual consumer of the operator layer
<code>campf1r3</code> (the repository)	The workspace: <code>kindling-norm</code> ’s <code>Hash32</code> and <code>blake2b_256</code> , <code>campf1r3-core</code> , <code>f1r3x</code> ; no external dependencies

1.3 Nomenclature

Term	Meaning in this document
<i>term</i>	A <code>f1r3comb_term::Term</code> : a drop-free normal form under presentation A, a flat multiset of atoms [2, Req. 4.4]
<i>name id</i>	A <code>NameId(u32)</code> : the in-run index of a name modulo $\equiv_{\mathcal{N}}$
<i>row</i>	One atom occurrence in the machine: a shape and a fixed-width tuple of name ids
<i>row ref</i>	(<code>Shape, u32</code>): a row’s position in its shape table; the machine-level occurrence identity of [1, Def. 8.1]
<i>incidence</i>	The family M_j^A of [1, Def. 7.1]; here implicit in the shape tables
<i>species</i>	A shape with a tuple of name ids [1, Def. 4.1]; a row’s value, forgetting its position
<i>marking</i>	A finite multiset of species; a basis vector $ \gamma\rangle$ of the state space
<i>redex</i>	A rule together with the row refs it consumes, in premise order
<i>step</i>	The firing of one independent set of redexes
<i>width</i>	The size of a step
<i>grading</i>	The function W of Erratum 9.1

The product name is F1R3COMB-MAT, written `f1r3comb-mat`, `f1r3comb-par` and `f1r3comb-`

`op` in crate names and `comb-mat`, `comb-width` and `comb-op` in F1R3X subcommands. The name is provisional and MAY be changed before M0 without affecting any requirement.

1.4 Goals

1. A total, lossless, round-trippable map from F1R3COMB terms to the matrix representation of [1, §7], and back.
2. Redex finding as joins on the incidence, derived mechanically from the rule table F1R3COMB already requires [2, Req. 4.7], with three independent finders that MUST agree.
3. A bulk-synchronous machine whose step is a maximal independent set, defined bit-exactly, so that the host sequential, host threaded and any future device backend produce identical traces.
4. Instrumentation sufficient to run the width measurement that gates device work [1, Obl. 13.7].
5. On the destructor-free sublattice, the transfer operator over reachable markings, its triangular structure, its closure, and the pre-image primitive the logic's modality needs [1, Rem. 11.1].
6. The F1R3COMB portability envelope: no external dependencies, `forbid(unsafe_code)`, `wasm32-unknown-unknown-clean`, bit-identical across hosts and core counts.

1.5 Non-goals

1. **Device code.** No GPU kernel is specified for the workspace. Section 12.4 specifies the backend contract a device implementation MUST meet and places it outside the workspace, because every device API is an external dependency. It is gated on Obligation 13.10.
2. **Presentation B.** The matrix representation is A-only; this is forced, not chosen ([1, Rem. 3.1]; Decision 2.1).
3. **Weights.** Boolean and natural-number semirings only. The **Semiring** trait of Section 5 is the seam where the weighted readings attach, and nothing here depends on them.
4. **The logic.** The operator layer exposes T , its closure and $T^\top \chi$. It MUST NOT implement satisfaction, characteristic formulae or model checking, which are [4]'s.
5. **The history unfolding.** Triangularity with `e` present via the universal cover [1, §6.1, route (1)] is deferred.
6. **Garbage collection of the intern table.** Deferred ([1, Obl. 13.3]; Requirement 11.1); inert rows are counted, not collected.

2 Position relative to F1R3COMB

2.1 Two machines, one term type

F1R3COMB v0.2 runs a term by instantiating CAMPF1R3: an interleaving machine, one comm per event, races resolved by the store's order key [2, Dec. 8.1]. F1R3COMB-MAT runs the same term by maximal progress: every step fires a maximal set of pairwise disjoint redexes. The two are different resolvers for the same nondeterminism [1, Rem. 8.5], and neither is a refinement of the other. What they share is fixed here.

	F1R3COMB machine (f1r3comb-run)	F1R3COMB-MAT (f1r3comb-par)
Input	Term	Term
State	CAMPF1R3 store keyed by Hash32	shape tables of NameId rows
Name identity	content hash of the encoding	in-run structural intern, content hash at the boundary
Event	one comm	one step, a maximal independent set
Resolver	store order key	priority function of Definition 7.3
Rule source	the shared rule table	the shared rule table
Output	a scene (Term)	a scene (Term)

2.2 Decisions inherited and decisions taken

Decision 2.1 (Presentation A only). F1R3COMB-MAT implements presentation A. With F1R3COMB’s **presentation-b** feature enabled, the three crates **MUST** compile to stubs whose entry points return **comb-mat-presentation**, so that a workspace-wide feature unification does not break the build. Under B a message payload is a process, rows are ragged, and requirement (G1) of [1, §2.1] fails by construction.

Decision 2.2 (A store row holds the quotation of its payload). In the term type, $q(a, p)$ carries a process, and F1R3COMB’s reversal of v0.1 on this point [2, Dec. 4.9, Rem. 4.10] stands. In the matrix representation, the row for $q(a, p)$ is $(q, \text{id}(a), \text{id}(@p))$. The two statements do not conflict, and this decision records why:

1. the row is a *representation*, not a sort: under A, $@-$ is injective on drop-free normal forms [4, Prop. 2.10], so storing $\text{id}(@p)$ loses nothing and export recovers p exactly;
2. the m/q distinction the logic turns on [4, Rem. 2.5] is carried by the *shape tag*, which is preserved; a q row and an m row with equal id tuples are different species;
3. the matrix representation is not a term type, and **MUST NOT** be offered to the logic tooling as one (Requirement 3.13).

The effect is the one [1, Rem. 3.1] wants: every row is a fixed-width tuple of integers, and the **quote** rule is a field copy (rule class (a)).

Decision 2.3 (Decode by arena read). [2, Req. 8.3] forbids the F1R3COMB machine to realise **eval** by projecting the payload out of the name, because the short-circuit is unsound under B. F1R3COMB-MAT realises it exactly so — a read of the intern arena at v — and is entitled to, because it is A-only (Decision 2.1) and because interning applies drop elimination before any component is stored (Requirement 3.4). The crate **MUST** carry a test asserting, for every name in a corpus, that the arena entry equals the atom multiset of the smart-constructed $*v$ in **f1r3comb-term**.

Decision 2.4 (One rule table). The rule table [2, Req. 4.7] **MUST** live in **f1r3comb-term** (module **rules**), not in **f1r3comb-run**, and both machines **MUST** be dispatches over it. Joins, conclusions, rule classes and the grading of Erratum 9.1 are all derived from the table; no rule is special-cased in F1R3COMB-MAT’s control flow. This is a request of F1R3COMB (Section 16, item 6).

Decision 2.5 (Shortcuts are supported). With F1R3COMB’s **shortcuts** feature, the shape set grows to fifteen. F1R3COMB-MAT **MUST** support it through the rule table alone. Draft 3 names **cons_e** and **cons_{fw}** but does not state their rules [3, Rem. 3.10, Rem. 6.7]; this document fixes them as the completions consistent with [2, Req. 7.12]’s arities:

$$\text{cons}_e(a, f) \mid m(a, u) \longrightarrow m(f, @e(u)), \quad \text{cons}_{fw}(a, b, f) \mid m(a, u) \mid m(b, v) \longrightarrow m(f, @fw(u, v)).$$

Section 16 asks draft 3 to state them.

3 The representation

3.1 Names and the intern table

[1, Def. 4.3] realises $\mathcal{N}$ as a bijection between integers and canonical component multisets. The Rust type is:

```
#[derive(Copy, Clone, PartialEq, Eq, PartialOrd, Ord, Hash, Debug)]
pub struct NameId(pub u32);

#[derive(Copy, Clone, PartialEq, Eq, PartialOrd, Ord, Hash, Debug)]
pub struct Row { pub shape: Shape, pub args: [NameId; 4] } // unused slots = NameId::PAD

pub struct InternTable {
    arena: Vec<Row>, // components of every name, contiguous
    span: Vec<(u32, u32)>, // NameId -> (offset, len) into arena
    key: HashMap<Box<[Row]>, NameId>, // structural key -> id (std only)
    depth: Vec<u32>, // quotation depth, for canonical export
    chash: Vec<Option<Hash32>>, // content hash, computed lazily
    budget: NameBudget,
}
```

Definition 3.1 (Structural key). The key of a name is the sorted slice of its component rows, sorted by `Row`’s derived order. By [4, Def. 2.9] components are a complete invariant, and by induction on quotation depth two names receive the same id iff they are $\equiv_{\mathcal{N}}$ ([4, Prop. 2.10]).

Requirement 3.2 (The hot path is hash-free). `quote` (insert-or-get by structural key) and `drop` (arena read) MUST NOT compute BLAKE2b. The content hash of `F1R3COMB` [2, Req. 4.12] is computed only at the boundary — export, tracing, canonical relabelling — memoised per id in `chash`. This is [1, Obl. 13.2] discharged: the canonical, run-stable hash the note asks for already exists, it is `F1R3COMB`’s encoding hash, and the machine does not need it to run.

Requirement 3.3 (Key and hash agree). For all ids i, j : $i = j$ iff `chash(i) == chash(j)`, modulo BLAKE2b collision. A property test MUST assert this on every corpus run, and the debug build MUST assert it whenever two ids’ hashes are both materialised. A collision is a `comb-mat-hash-collision` diagnostic, never a merge.

Requirement 3.4 (Interning normalises). Components entering the arena MUST be drop-free, which import guarantees because `Term` is ([2, Req. 4.4]), and constructor firing guarantees because every class (b) conclusion is built from rows. The `cons` rule’s union `drop(u)  $\uplus$  drop(r)` MUST be computed as a multiset merge of two sorted arena spans, so the result is its own key without a further sort.

Remark 3.5 (Interning order is a gauge freedom, and is fixed anyway). [1, Rem. 4.4] observes that ids depend on insertion order and that nothing observable may depend on them. That remains true of the semantics. But Requirement 15.4 needs the machine’s internal choices to be reproducible, and those choices read row positions and ids. `F1R3COMB-MAT` therefore fixes insertion order (Requirement 8.1) so that ids are reproducible, and separately forbids ids from crossing any external boundary (Requirement 3.6). The gauge is fixed for determinism and hidden for semantics.

Requirement 3.6 (Ids never leave the process). No trace event, report, diagnostic, export or public API return value MAY contain a `NameId` or a row position, except the `.cmat` format of Section 3.4, which uses canonically relabelled ids. Externally a name is its `Hash32`.

Requirement 3.7 (Budget). [2, Req. 8.8]’s budget carries over: maximum distinct names, maximum components per name, maximum arena length. Exceeding any MUST halt the step before it commits, with `comb-name-budget` naming the rule and the redex’s consumer as a content hash.

3.2 State: shape tables

Definition 3.8 (State). A state is one table per enabled shape. The table for shape A of arity k is k columns of `NameId`, all of equal length; row i of the table is the i -th occurrence of A .

```
pub struct ShapeTable { pub cols: [Vec<NameId>; 4], pub len: u32 } // cols[j] unused for j >=
    arity
pub struct State { pub tables: [ShapeTable; SHAPES] } // SHAPES = 13, or 15
    with shortcuts
```

Requirement 3.9 (Structure of arrays). Columns **MUST** be separate contiguous arrays. This is (G1) of [1, §2.1] and it is what makes the incidence implicit (Definition 3.10). An array of `Row` structs **MUST NOT** be used for the state; it is used only for the arena, where rows of mixed shape must be contiguous for decode.

Definition 3.10 (Incidence, implicit). The Boolean matrix M_j^A of [1, Def. 7.1] has exactly one nonzero per row, at column `tables[A].cols[j][i]`. Its CSR form is therefore `indptr = 0, 1, ..., n` and `indices = cols[j]`. The column array *is* the matrix. It **MUST NOT** be materialised separately in the production path; the generic finder of Section 6 **MAY** materialise it, through the `Csr::from_column` view, for validation.

3.3 Import and export

Requirement 3.11 (Import). `State::import(&Term, &mut InternTable)` **MUST** flatten the term’s components in encoding order and push one row per atom, interning every name argument bottom-up with an explicit stack — the workspace’s no-recursion rule [2, Req. 7.1] applies. An import memo keyed by `Hash32` **MAY** be used to skip re-interning a name already seen; it **MUST** agree with the structural key (Requirement 3.3). The payload of `q` is interned as its quotation (Decision 2.2).

Requirement 3.12 (Export). `State::export(&InternTable) -> Term` **MUST** rebuild the term through `f1r3comb-term`’s smart constructors, rendering a `q` row’s second argument by `drop`. Import followed by export **MUST** be the identity on encodings, and export followed by import **MUST** be the identity on markings. Both are property tests.

Requirement 3.13 (Not a term type). `State`, `Row` and `InternTable` **MUST** be documented as a machine representation. [2, Req. 9.2]’s public API for the logic is `f1r3comb-term`’s, and `F1R3COMB-MAT` **MUST NOT** expose a second one.

3.4 The canonical export format

A term’s matrix representation is worth a file of its own, for three consumers: a device backend in another process, the Python reference, and golden tests. In-run ids are not canonical, so the file relabels them.

Definition 3.14 (Canonical relabelling). Order the names reachable from a state by quotation depth, ties broken by content hash, and assign ids $0, 1, \dots$ in that order; every name’s components then refer only to smaller ids. Within each shape table, order rows lexicographically by relabelled id tuple.

Requirement 3.15 (`.cmat`). The `.cmat` encoding **MUST** be: a header (magic `CMAT`, format version, presentation, `shortcuts` flag, shape count); the name table in relabelled order, each entry a `LEB128` component count followed by rows, each row a shape tag from [2, §4.5] and `LEB128` ids; then the state, per shape a `LEB128` row count followed by columns. It **MUST** satisfy: for terms $P \equiv Q$ the `.cmat` encodings are byte-equal. With `F1R3COMB`’s canonicity theorem [2, Obl. 5.8] this makes the file a function of the source congruence class.

Requirement 3.16 (Python interoperability). `f1r3comb-check` MUST ship a loader for `.cmat` in the Python reference’s representation (a `NameTable` and a `State`), as a script outside the Rust build, so that Obligation 13.3 can run the Rust and Python machines on one input.

4 The rule table, as F1R3COMB-MAT reads it

4.1 The table

```
pub enum Var { Slot(u8), Payload(u8) }           // consumer’s arg j; premise j’s payload
pub enum Premise { Loud { subject: u8 }, AnyAt { subject: u8 } } // AnyAt: m or q (fw only)
pub enum Conclusion {
  Atoms(&'static [(Shape, &'static [Var])]), // class (a)
  Encode { deliver: Var, node: NodeTemplate }, // class (b): m(deliver, @node)
  Decode { payload: Var }, // class (c): *payload
}
pub enum NodeTemplate { Union(Var, Var), Atom(Shape, &'static [Var]) }
pub struct RuleSpec {
  pub name: &'static str, pub consumer: Shape,
  pub premises: &'static [Premise], pub conclusion: Conclusion, pub class: RuleClass,
}
```

The rows, for the thirteen-shape calculus of [3, Def. 3.3] and [2, Def. 4.6]:

Rule	Consumer	Premises	Conclusion
dist	$d(a, b, c)$	loud at a	$m(b, v_0) \mid m(c, v_0)$
kill	$k(a)$	loud at a	0
forward	$fw(a, b)$	loud at a	$m(b, v_0)$
bind _l	$b_l(a, b)$	loud at a	$fw(v_0, b)$
bind _r	$b_r(a, b)$	loud at a	$fw(b, v_0)$
synch	$s(a, b, c)$	loud at a	$fw(b, c)$
eval	$e(a)$	loud at a	decode v_0
quote	$fw(a, b)$	quiet at a	$m(b, v_0)$
make _l	$cons(a, b, c)$	loud at a, b	$m(c, @(*v_0 \mid *v_1))$
make _m	$cons_m(a, b, c)$	loud at a, b	$m(c, @m(v_0, v_1))$
make _d	$cons_d(a, b, c, f)$	loud at a, b, c	$m(f, @d(v_0, v_1, v_2))$
make _s	$cons_s(a, b, c, f)$	loud at a, b, c	$m(f, @s(v_0, v_1, v_2))$
make _e	$cons_e(a, f)$	loud at a	$m(f, @e(v_0))$ shortcuts
make _{fw}	$cons_{fw}(a, b, f)$	loud at a, b	$m(f, @fw(v_0, v_1))$ shortcuts

Remark 4.1 (`fw` has two rows). `fw` is the one consumer with two rules, one per datum kind. F1R3COMB’s machine merges them into one `consume` with an `accepts_quiet` pattern [2, §8.2]; F1R3COMB-MAT keeps them as two rows because they join against different tables (`m` and `q`). The race $fw(a, b) \mid m(a, v) \mid q(a, p)$ [2, Req. 8.6] then appears as two redexes sharing the `fw` row, which the conflict structure resolves. The quiet/loud distinction therefore lives in the join, not in a matcher, and Requirement 4.2 makes that exact.

Requirement 4.2 (Only a forwarder reads a store). No rule other than `quote` MAY join against the `q` table. A test MUST assert that for every consumer shape other than `fw`, a state consisting of that consumer and a `q` row at its subject has no redex. This is the inertness of an ungated gate [3, Lem. 6.5] in F1R3COMB-MAT’s terms.

4.2 Rule classes

Requirement 4.3 (Class is data). Each row MUST carry its class from [1, §5.1]: (a) seven rules whose conclusions are fixed multisets over premise names (including `quote`, by Decision 2.2); (b) the constructors, which encode; (c) `eval`, which decodes. A test MUST assert that the class recorded agrees with the `Conclusion` variant, and that exactly one rule is class (c).

4.3 Join plans

Definition 4.4 (Join plan). The join plan of a rule is the list of pairs (consumer slot, premise table) read off its premises: premise j joins column `subjectj` of the consumer’s table against column 0 of the `m` table (loud) or of the `q` table (quiet). A rule with k premises has a k -way join, $k \leq 3$.

Requirement 4.5 (Plans are derived). Join plans **MUST** be computed from the rule table at start-up (or `const`-evaluated) and **MUST NOT** be written by hand. The table of [1, §7.1] is then a theorem about the derivation, and a test **MUST** check it: seven one-premise loud plans, one one-premise quiet plan, two (four with `shortcuts`) two-premise plans, two three-premise plans.

5 Sparse algebra

No external dependency is available, so the crate carries its own small sparse layer. It is used by the generic finder, the reference conflict product and the operator layer; the production machine uses only its counting-sort primitive.

```
pub trait Semiring: Copy + Eq + core::fmt::Debug {
    const ZERO: Self; const ONE: Self;
    fn add(self, o: Self) -> Self; fn mul(self, o: Self) -> Self;
}
#[derive(Copy, Clone, PartialEq, Eq, Debug)] pub struct Bool(pub bool);
#[derive(Copy, Clone, PartialEq, Eq, Debug)] pub struct Nat(pub u64); // checked; overflow
is an error

pub struct Csr<S: Semiring> { pub rows: u32, pub cols: u32,
                             pub indptr: Vec<u32>, pub indices: Vec<u32>, pub vals: Vec<S> }
impl<S: Semiring> Csr<S> {
    pub fn from_column(col: &[NameId], ncols: u32) -> Self; // the implicit incidence view
    pub fn transpose(&self) -> Self; // counting sort, 0(nnz + cols)
    pub fn spgemm(&self, rhs: &Self) -> Result<Self, Overflow>; // Gustavson
    pub fn spmv(&self, x: &SparseVec<S>) -> Result<SparseVec<S>, Overflow>;
}
```

Requirement 5.1 (Semantics of the layer). `spgemm` **MUST** be Gustavson’s row-by-row algorithm [9] with a dense accumulator reset per row, producing sorted column indices and no explicit zeros. `Nat` arithmetic **MUST** be checked; overflow is `comb-mat-overflow`, never a wrap. `Bool` **MAY** store no `vals` (a specialisation), provided the observable results are equal. The layer is correct by comparison with a dense reference on random matrices up to 64×64 , as a property test.

Requirement 5.2 (Bucketing primitive). `bucket(col: &[NameId]) -> Buckets` **MUST** return, for each name occurring in `col`, the ascending list of rows carrying it, as a CSR over the *occurring* names only (sorted, deduplicated), so that its cost is $O(n \log n)$ in the column length or $O(n)$ by radix sort, and never proportional to the size of the intern table. It is $(M_j^A)^\top$ restricted to its nonempty rows.

6 Redex finding

6.1 The redex

Definition 6.1 (Redex). A redex is `(rule: u8, consumer: u32, premises: [u32; 3])`: the rule’s index in the table, the consumer’s row in its table, and for each premise the row in the `m` or `q` table, padded. It is *enabled* if each premise row’s subject equals the corresponding consumer slot, and the premise rows are pairwise distinct when they are in the same table.

Definition 6.2 (Canonical key). The canonical key of a redex is the tuple (rule, consumer, premises) under lexicographic order. Redex sets are always delivered sorted by canonical key.

Erratum 6.3 (Symmetric matches are one redex). The reference enumerates ordered premise tuples subject only to distinctness of occurrences. When two premise positions of a constructor receive *equal atoms* — equal subject and equal payload, as in $\text{cons}(a, a, c) \mid \mathbf{m}(a, u) \mid \mathbf{m}(a, u)$ — the two orderings consume the same multiset and produce the same result, and the reference reports two redexes. They are one derivation: [1, Prop. 5.3]’s coefficient divides by $m_\alpha!$ for exactly this reason, and the reference’s count is 2 where the proposition’s is 1. The step is unaffected (the two conflict, and either may be chosen), and none of the E1–E4 families contain such a match, so `results.txt` is unaffected. But the operator layer computes the coefficient, and a resolver that picks one redex uniformly from the enumerated set — the reference’s `run_sequential` — double-weights symmetric matches, which is a semantic commitment [1, Obl. 13.1] did not intend.

Requirement 6.4 (Symmetry mask). Every finder `MUST`, in addition to the distinctness mask of [1, §7.1], drop an ordered premise tuple in which two positions carrying equal atoms appear with the later position at a lower row. The enumerated set is then in bijection with derivations, and its size is the `Nat` coefficient of [1, Prop. 5.3] summed over results.

6.2 Three finders

Requirement 6.5 (Three implementations, one answer). `f1r3comb-par` `MUST` provide three finders behind one trait, and `MUST` test that they return identical sorted redex sets on every state in the property corpus.

1. **Naive**: for each consumer row and each rule of its shape, nested loops over the premise tables. The oracle; no sparse code.
2. **Product**: per join plan, the Boolean product $M_j^C \cdot (M_0^{\mathbf{m}})^\top$ via `Csr::spgemm`, then the row-wise Cartesian product of the per-premise results with the distinctness and symmetry masks. The literal content of [1, §7.1], and of the reference’s `redexes_matrix`.
3. **Bucketed**: bucket the `m` and `q` subject columns once per step (Requirement 5.2); for each consumer row, look up its subject columns’ buckets and expand. The production finder. It computes the same product, specialised to operands with one nonzero per row.

Remark 6.6 (Why the product specialises). Every incidence matrix has exactly one nonzero per row, so $(M_j^C \cdot (M_0^{\mathbf{m}})^\top)[i, \cdot]$ is the bucket of the single name in row i of M_j^C . Gustavson’s inner loop degenerates to one bucket lookup. The **Product** finder is kept because it is the form in which the note’s claim is stated and the form a device backend will implement; the **Bucketed** finder is kept because it is the form that is fast on a host.

Requirement 6.7 (Order of output). All finders `MUST` return redexes sorted by canonical key. The **Bucketed** finder `MAY` generate in any order and sort, or generate in order by iterating rules, then consumer rows ascending, then bucket entries ascending, which yields sorted output without a sort.

6.3 Contention

Hazard 6.8 (The join is quadratic on a contended channel). [1, Rem. 7.2]: c consumers and μ messages on one name yield $c\mu$ redexes, measured as c^2 in E2. The step does not suffer — the independent set is a matching found in one step — but enumeration does.

Requirement 6.9 (Contention remedies preserve the step). A finder that enumerates a subset of the enabled redexes (a capped or worst-case-optimal join [11]) MAY be added behind the `JoinStrategy` enum, provided the step it drives is *identical* to the step driven by a complete finder under the same priority function. The standard way to meet this is to enumerate lazily in priority order per occurrence; any strategy MUST be differentially tested against `Bucketed` on the step trace, not merely on maximality. Which strategy is right at scale is open [1, Rem. 7.2]; v0.1 ships `Bucketed` only and reports enumerated-to-fired ratios (Section 11).

7 Conflict and selection

7.1 The conflict structure

Definition 7.1 (Conflict). Let B be the redex-by-row-ref incidence. Two redexes conflict iff they share a row ref, i.e. iff $(BB^\top)_{rs} \neq 0$ with $r \neq s$ [1, Def. 7.3].

Requirement 7.2 (Hypergraph, not graph, in production). The production machine MUST represent conflict by $B^\top$ — for each row ref, the redexes consuming it — and MUST NOT materialise $BB^\top$, whose size is the sum of squared row-ref degrees. A reference `conflict_graph` computing $BB^\top$ by `Csr::spgmm`, as the Python reference does, MUST exist and be tested to induce the same selection.

7.2 The priority function

Definition 7.3 (Priority). Let `mix64` be the SplitMix64 finaliser [10]:

```
fn mix64(mut z: u64) -> u64 {
    z = (z ^ (z >> 30)).wrapping_mul(0xbf58476d1ce4e5b9);
    z = (z ^ (z >> 27)).wrapping_mul(0x94d049bb133111eb);
    z ^ (z >> 31)
}
```

For run seed σ , step number t , and a redex with canonical key (r, c, p_0, p_1, p_2) , let $h_0 = \text{mix64}(\sigma)$, $h_1 = \text{mix64}(h_0 \oplus t)$, $h_2 = \text{mix64}(h_1 \oplus r)$, $h_3 = \text{mix64}(h_2 \oplus c)$, and $h_{4+i} = \text{mix64}(h_{3+i} \oplus p_i)$ over the rule's premises only. The priority is the pair $(h_{\text{last}}, \text{canonical key})$ under lexicographic order; smaller is earlier.

The priority is a function of the seed, the step and the redex's canonical key, not of the order in which a finder happened to produce it. The canonical key breaks hash ties, so the order is total.

7.3 The step

Definition 7.4 (The maximal-progress step). Given the enabled redexes R of a state and the priority order, the step S is the *greedy maximal independent set*: visit R in priority order, and take a redex iff none of its row refs has been taken.

Proposition 7.5 (Greedy equals rounds). *Let S' be computed in rounds: in each round, take every undecided redex that precedes, in priority order, every undecided redex it conflicts with; then mark its conflicting redexes decided. Then $S' = S$.*

Sketch. Induction on priority order. A redex taken in some round has, at that round, no undecided earlier conflicting redex, and every earlier conflicting redex was decided either by being taken — impossible, since it would have excluded this one — or by being excluded, in which case the greedy scan also excludes it and reaches this redex with its rows free. Conversely each greedy choice is a local minimum once its earlier neighbours are decided. This is the determinism result of [7]: the random-priority parallel algorithm computes exactly the lexicographically-first maximal independent set. $\square$

Requirement 7.6 (The step is Definition 7.4, on every backend). Every backend **MUST** compute exactly the set of Definition 7.4. The host sequential backend **MAY** run the greedy scan; the host threaded backend and any device backend **SHOULD** run the rounds of Proposition 7.5, whose expected round count is logarithmic [7]. Because the result is a function of the redex set and the priority, core count cannot change it. This is what makes [1, P4]’s exit criterion — identical step traces under a fixed seed — achievable rather than aspirational, and it corrects [1, Rem. 8.7]’s suggestion that the device should use “the standard” algorithm while the reference uses a different one: with shared priorities they compute the same set.

7.4 Resolvers

```
pub enum Resolver {
    MaximalProgress,      // the greedy maximal step; the default
    SingleByPriority,      // fire only the first redex in priority order
    SingleUniform,        // fire one redex chosen by mix64 over the enumerated set
}
```

Requirement 7.7 (Resolvers are named, and the default is explicit). The resolver **MUST** be a configuration value recorded in every report and trace header. **SingleUniform** reproduces the reference’s **run_sequential** under Requirement 6.4’s corrected enumeration, and is the baseline [1, Obl. 13.1] compares against. **SingleByPriority** is the sequential machine whose trace linearises a maximal-progress run (Obligation 13.4). No resolver **MAY** be selected implicitly by backend.

Requirement 7.8 (Races are visible). [2, Req. 8.6] requires every race resolution to be a trace event. Under maximal progress the analogue is: for each redex in S , the redexes that conflicted with it and were excluded because of it. At trace level **races** the machine **MUST** emit these, rendered by content hash; at lower levels it **MUST** emit their count. The full list can be quadratic (Hazard 6.8), which is why it is a level.

8 Firing

Requirement 8.1 (Order of effects). A step **MUST** commit as follows.

1. Sort S by canonical key.
2. In that order, compute each redex’s conclusion. Class (a) reads its consumer and premise rows. Class (c) reads the arena span of v_0 . Class (b) builds its node, then **quotes** it; names new to the table receive ids in this order.
3. Remove every consumed row from its table by *stable* compaction.
4. Append every produced row to its table, in the order of step 2 and, within one redex, in conclusion-template order (arena order for class (c)).

Parallel backends **MAY** compute conclusions concurrently, but new ids **MUST** be assigned as if step 2 were sequential, for example by a deterministic prefix sum over per-redex new-name counts followed by a reconciliation pass that merges duplicates to the earliest redex’s id.

Proposition 8.2 (Determinism). *Given the imported state, the seed and the configuration, the sequence of states, as shape tables of in-run ids, is determined.*

Sketch. Import is a function of the term’s encoding order. Given a state, the redex set is determined (Requirement 6.7), the priority is determined (Definition 7.3), the step is determined (Requirement 7.6), and by Requirement 8.1 so are the new ids and the successor tables. $\square$

Requirement 8.3 (Budget before commit). All budget checks (Requirement 3.7) **MUST** happen in step 2, before step 3 mutates anything, so that a halted step leaves the previous state intact and exportable.

Requirement 8.4 (Soundness is checked, not assumed). [1, Thm. 8.3] makes any linearisation of a step a valid interleaving. The debug build **MUST** assert the diamond property on each step it fires by re-finding redexes after each individual firing in canonical order and checking that the rest of S remains enabled; release builds **MUST NOT** pay for it.

9 The grading, and what it bounds

The note uses draft 3’s three lattice invariants as the finiteness conditions of the operator [1, Thm. 6.1]. F1R3COMB-MAT asserts them at run time and in the operator layer, so they have to be right. Two are not.

Erratum 9.1 (The atom-count lemma). [3, Lem. 9.2] states that every rule but **eval** strictly decreases the number of non-message atoms. Its own proof notes the exception: $\mathbf{b}_l$, $\mathbf{b}_r$ and $\mathbf{s}$ consume themselves and produce a $\mathbf{fw}$, so they leave the count unchanged. The consequence is concrete. For

$$T = \mathbf{b}_l(a, b) \mid \mathbf{m}(a, v) \mid \mathbf{m}(v, x),$$

with one non-message atom, $T \longrightarrow \mathbf{fw}(v, b) \mid \mathbf{m}(v, x) \longrightarrow \mathbf{m}(b, x)$ in two steps, so the bound of [3, Prop. 9.9] (“every reduction sequence has length at most the number of non-message atoms”) fails, and so does the nilpotency index $T^{N+1} = 0$ of [1, Thm. 6.1(ii)]. On 1,955 random firings with the Python reference, the non-message count failed to decrease 748 times.

Both conclusions survive. Define the grading

$$\mathbf{W}(\mathbf{b}_l) = \mathbf{W}(\mathbf{b}_r) = \mathbf{W}(\mathbf{s}) = 2, \quad \mathbf{W}(\mathbf{m}) = 0, \quad \mathbf{W}(A) = 1 \text{ for every other shape,}$$

extended additively to states. Every rule but **eval** decreases $\mathbf{W}$ by at least 1: $\mathbf{d}$, $\mathbf{k}$, $\mathbf{fw}$ and the constructors by 1; $\mathbf{b}_l$, $\mathbf{b}_r$, $\mathbf{s}$ from 2 to 1; **quote** from 2 to 0. (The shortcut constructors are ordinary shapes of weight 1 and decrease it by 1.) Hence without $\mathbf{e}$ every reduction sequence from T has length at most $\mathbf{W}(T) \leq 2N$, strong normalisation stands, and the operator is nilpotent with $T^{\mathbf{W}(T)+1} = 0$. On the same 1,955 firings, $\mathbf{W}$ decreased every time.

Erratum 9.2 (The message-count invariant). [1, Thm. 6.1(iii)] states that without $\mathbf{d}$ and $\mathbf{e}$ the message count is non-increasing. [3, Lem. 9.3], which it cites, is careful to say **quote** “adds one but consumes a $\mathbf{q}$ ”; the theorem drops the qualification. $\mathbf{fw}(a, b) \mid \mathbf{q}(a, p) \longrightarrow \mathbf{m}(b, @p)$ raises the message count from 0 to 1 (314 times in the same random run). The invariant that holds is on *messages plus stores*: without $\mathbf{d}$ and $\mathbf{e}$ it is non-increasing (zero violations in the same run), and occupation numbers of $\mathbf{m}$ and $\mathbf{q}$ species together are bounded by those of T .

Requirement 9.3 (The grading is derived and asserted). $\mathbf{W}$ **MUST** be computed from the rule table: a shape’s weight is 1 plus the weight of the heaviest non-message atom any of its rules produces from it, which yields the values above and adapts to any future rule. The debug build **MUST** assert, on every non-**eval** firing, that $\mathbf{W}$ decreases, and on every step without $\mathbf{d}$ or $\mathbf{e}$ firings, that messages plus stores do not increase. Both are also property tests.

10 The operator layer

f1r3comb-op realises [1, §5]: the transfer operator T over markings, where it is finite.

10.1 Species and markings

```
#[derive(Copy, Clone, PartialEq, Eq, PartialOrd, Ord, Hash)] pub struct SpeciesId(pub u32);
pub struct SpeciesTable { /* (Shape, [NameId; 4]) <-> SpeciesId */ }
#[derive(Clone, PartialEq, Eq, Hash, PartialOrd, Ord)]
pub struct Marking(pub Box<[(SpeciesId, u32)]>); // sorted by SpeciesId, counts > 0
```

Requirement 10.1 (Markings are canonical). A marking **MUST** be the sorted run-length encoding of a state’s rows by species. Two states have equal markings iff they are structurally congruent ([1, F2], [4, Def. 2.9]). A marking is materialised as a **State** for redex finding by expanding counts into rows in species order, so that the finders of Section 6 are reused unchanged.

10.2 Successors: one column of T , lazily

Requirement 10.2 (Successors). `successors(&Marking) -> Vec<(RuleId, Marking, Nat)>` **MUST** return, for each rule and each resulting marking, the number of derivations producing it — the matrix element $\langle \gamma' | H_r | \gamma \rangle$ of [1, Prop. 5.3] — computed by enumerating redexes under Requirement 6.4, firing each on a copy, and grouping. It **MUST** work in the presence of **e** and of constructors: it is the frontier of [1, §6.1, route (2)], which needs no finiteness.

Remark 10.3 (Rows versus occurrences). Expanding a marking into rows gives occurrences of one species distinct row refs, and the enumeration counts ordered choices among them. Grouping by result and applying the symmetry mask gives exactly $\prod_{\alpha} (\gamma(\alpha))_{m_{\alpha}} / \prod_{\alpha} m_{\alpha}!$: the falling factorial comes from distinct row refs, and the division from the mask. A property test **MUST** compare `successors` against the closed formula on random small markings.

10.3 Exploration and the materialised operator

Requirement 10.4 (Reachable markings). `explore(&Marking, ExploreBudget) -> Reach` **MUST** compute the set of markings reachable from the initial one by breadth-first search over `successors`, with a budget on markings and on names. On a term with no **e** row and no rule that can produce one — decidable from the rule table and the initial state’s arena closure — the reachable set is finite by Erratum 9.1 and König’s lemma, and exploration **MUST** terminate within the budget or report `comb-op-budget`. On any other term, `explore` **MUST** refuse with `comb-op-destructor` unless the caller supplies an explicit depth bound, in which case the result is marked *truncated* and no triangularity or closure claim is made for it.

Remark 10.5 (Constructors do not prevent finiteness). [1, Thm. 6.1(i)] says constructors make the *name* set generated rather than given. That is a statement about the species basis, not about reachable markings. Without **e** the reachable marking set is finite with or without constructors, because **W** bounds derivation length and each marking has finitely many successors. So the operator layer covers the whole **e**-free sublattice, and constructors only mean that the species table grows during exploration.

Requirement 10.6 (Materialising T). `Reach::operator::<S>() -> Csr<S>` **MUST** return T over the reachable markings, indexed by the marking order of Definition 10.7, with $T[\gamma', \gamma]$ the sum over rules of the successor counts (**Nat**) or their support (**Bool**). It is the object [1, §7] calls “astronomically large, never materialised” in general; here it is materialised only on explored, finite, **e**-free reachable sets.

Definition 10.7 (Marking order). Order markings by **W** descending, ties broken by the marking’s derived order.

Requirement 10.8 (Triangularity is asserted). On an **e**-free reach, T in the order of Definition 10.7 **MUST** be strictly lower triangular in the (column $\rightarrow$ row) convention used here —

every nonzero $T[\gamma', \gamma]$ has γ' strictly later than γ — and the crate MUST assert it. This is [1, Thm. 6.1(ii)] under Erratum 9.1; the note’s “upper” is the transpose convention and the crate MUST document which it uses.

Requirement 10.9 (Closure). `closure::<S>() -> Csr<S>` MUST compute $\sum_{k=0}^{W_0} T^k = (\mathbf{1} - T)^{-1}$, where W_0 is the grading of the initial marking, as the product $\prod_{i=0}^{\lceil \log_2(W_0+1) \rceil - 1} (\mathbf{1} + T^{2^i})$ computed by repeated squaring — $\lceil \log_2(W_0 + 1) \rceil$ squarings, correcting [1, Cor. 6.2]’s $\lceil \log_2 N \rceil$ — and MUST assert $T^{W_0+1} = 0$. Over **Bool** it is reachability; over **Nat** it counts derivations, with checked arithmetic.

Requirement 10.10 (The primitive the logic needs). `pre::<S>(&SparseVec<S>) -> SparseVec<S>` MUST return $T^\top \chi$: for a characteristic vector χ over reachable markings, the vector whose entry at γ sums χ over the successors of γ . This is the modality of [4] as a matrix-vector product [1, Rem. 11.1]. It is exposed and tested; the logic that uses it is not implemented here (Section 1.5).

Hazard 10.11 (Reachable sets are exponential). The reachable set of an **e**-free term can be exponential in its size even though every derivation is short: the E1 family, k independent forwarder chains of length d , has $(d+1)^k$ reachable markings and derivations of length kd . The operator layer is an analysis tool for small terms and for the logic, not an execution strategy, and the budget is not optional.

11 Instrumentation and the width gate

```
pub struct StepReport { pub step: u64, pub enumerated: u64, pub fired: u32,
                        pub new_names: u32, pub rows_after: u64, pub per_rule: [u32; RULES] }
pub struct RunReport { pub steps: u64, pub fired: u64, pub widths: Histogram,
                       pub enumerated: Histogram, pub names: u64, pub arena: u64,
                       pub inert_rows_final: u64, pub halted: Option<Diag>, pub config:
RunConfig }
```

Requirement 11.1 (Reports). Every run MUST produce a **RunReport**, and every step a **StepReport** at trace level **steps** or above. *Inert rows* are rows that no rule could consume given the current subject columns — a consumer whose subject bucket is empty, a message or store whose subject no consumer carries — counted by the name-degree test of [1, Obl. 13.3], not removed.

Requirement 11.2 (The width measurement). `f1r3x comb-width files...` MUST compile each file with F1R3COMB, run it under **MaximalProgress** and under **SingleByPriority** from the same seed, and report per program: rewrites, parallel steps, the ratio, the width histogram, enumerated-to-fired ratio, name growth, and the lattice point [2, Req. 9.1]. `--seeds` MUST average over seeds exactly as the reference’s **measure** does. This is the instrument for [1, P2] and Obligation 13.10.

Requirement 11.3 (Subcommands). F1R3x gains, alongside F1R3COMB’s **comb**, **comb-hash**, **comb-run** and **comb-lattice**:

<code>f1r3x comb-mat file</code>	Compile and write the <code>.cmat</code> (Requirement 3.15); <code>--dump</code> prints the relabelled tables
<code>f1r3x comb-run -machine=matrix</code>	Run under F1R3COMB-MAT; <code>--seed</code> , <code>--resolver</code> , <code>--cores</code> , <code>--trace</code>
<code>f1r3x comb-width files</code>	Requirement 11.2
<code>f1r3x comb-op file</code>	Explore, report reachable count, nilpotency index, and the terminal markings; <code>--budget</code>

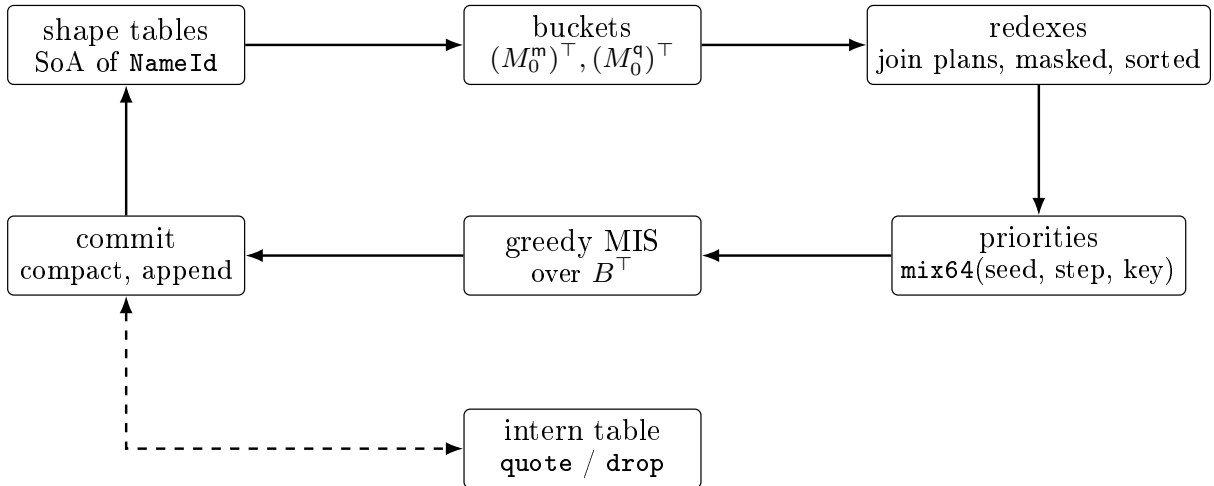
12 Architecture

12.1 The crates

Crate	What it is	Depends on
<code>f1r3comb-mat</code>	Intern table, rows, shape tables, import, export, <code>.cmat</code> , the sparse layer and <code>Semiring</code>	<code>f1r3comb-term</code>
<code>f1r3comb-par</code>	Join plans, the three finders, conflict, priority, selection, firing, backends, reports	<code>f1r3comb-mat</code>
<code>f1r3comb-op</code>	Species, markings, successors, exploration, T , closure, <code>pre</code>	<code>f1r3comb-par</code>
<code>f1r3comb-check</code> (extended)	Python parity, trace certification, differential tests against <code>f1r3comb-run</code> and <code>B3LL0WS</code>	all

Requirement 12.1 (Independence). `f1r3comb-mat` MUST be usable without `f1r3comb-par`, so that a device host process can load and emit `.cmat` without the host machine; `f1r3comb-par` MUST be usable without `f1r3comb-op`; and none of the three MAY depend on `f1r3comb-lower` or `f1r3comb-run`. `F1R3COMB-MAT` takes a `Term`, however it was produced.

12.2 One step



This is [1, Fig. 2] with two changes: the conflict product is replaced by its transpose (Requirement 7.2), and a priority stage is made explicit, because it is what the backends must agree on.

12.3 Backends

```

pub trait Backend {
    fn find(&mut self, st: &State, plans: &Plans, out: &mut RedexSet);
    fn select(&mut self, r: &RedexSet, prio: &PrioFn, out: &mut Vec<u32>); // indices into r
    fn commit(&mut self, st: &mut State, it: &mut InternTable, r: &RedexSet,
              sel: &[u32]) -> Result<StepReport, Diag>;
}
pub struct HostSequential; // greedy scan; the reference
pub struct HostThreaded { pub cores: u32 } // std::thread::scope; rounds of Prop. 6.6
  
```

Requirement 12.2 (Threaded backend). `HostThreaded` MUST use only `std::thread::scope`, MUST partition work by contiguous ranges of consumer rows and of redexes, and MUST merge

results in range order. On `wasm32` it MUST fall back to `HostSequential`, reporting `cores = 1`. Its output MUST be bit-identical to `HostSequential`'s at every core count (Requirement 15.4).

12.4 The device seam

Requirement 12.3 (Device backends live outside the workspace). A device backend necessarily depends on a device API, which the workspace forbids [2, Req. 12.3]. It MUST therefore be a separate crate in a separate repository, implementing `Backend` against the `.cmat` layout, and MUST be accepted only if its step traces equal `HostSequential`'s under the same seed on the conformance corpus (Obligation 13.9). Its kernels are fixed by this document: bucketing is a sort by key; each join plan is a segmented expansion over buckets; masks are elementwise; priorities are `mix64` per redex; selection is the rounds of Proposition 7.5; commit is a stable compaction plus an append by prefix sum; encode is a batched insert-or-get; decode is a gather from the arena. No work on it MAY begin before Obligation 13.10 is discharged.

13 Correctness obligations and testing

Obligation 13.1 (Round trip). Requirement 3.12: import-export is the identity on encodings and export-import the identity on markings, for the `F1R3COMB` property generator's output and for random states over all enabled shapes.

Obligation 13.2 (Finder agreement). Requirement 6.5, generalising the reference's `validate`: on at least 10,000 random states over all enabled shapes, two to five names and two to twelve atoms — the reference's generator, widened — the three finders return identical sorted redex sets.

Obligation 13.3 (Python parity). On the E1–E4 families, the Rust machine MUST reproduce every column of `results.txt` except timing: atoms, parallel steps, rewrites, sequential steps, maximum width, maximum enumerated redexes, and the ratio. These are seed-independent for the four families, so the check is exact, and it is a golden file. They are also laws, and MUST be tested as laws over their parameters: E1 takes exactly d steps and kd rewrites; E2 one step, c rewrites and c^2 enumerated; E3 $h + 1$ steps and $2^{h+1} - 1$ rewrites; E4 two steps and $2k$ rewrites. On random states, redex sets MUST agree with the Python reference up to the id bijection, via Requirement 3.16, as a development tool outside CI.

Obligation 13.4 (Trace certification). Every maximal-progress step MUST be certifiable: `f1r3comb-check` linearises it in canonical order and replays each firing with an independent single-redex rewriter written directly from [2, Def. 4.6] against `Terms` — not from the rule table — checking that each is a valid rewrite and that the final term equals the export of the step's result. This is [1, Thm. 8.3] tested against a second reading of the rules.

Obligation 13.5 (Differential against `F1R3COMB`'s machine and `B3LL0WS`). For corpus programs whose observations are resolver-independent (confluent on encoded observations), the quiescent observation of `F1R3COMB-MAT`, of `f1r3comb-run` and of `B3LL0WS` MUST agree, with observation as in [2, Obl. 10.4]: messages at encodings of source names only. For the rest, `F1R3COMB-MAT`'s final state MUST be certified by Obligation 13.4, since the three machines are entitled to different results.

Obligation 13.6 (Operator correctness). On random `e`-free terms under the exploration budget: `successors` matches the closed formula (Remark in Section 10); T is strictly triangular in the marking order; $T^{W_0+1} = 0$; the `Bool` closure equals BFS reachability; every terminal marking of the reach is reached by some maximal-progress run under some seed in a sample, and every maximal-progress run's final marking is terminal in the reach.

Obligation 13.7 (Determinism). The trace, the reports and the `.cmat` of the final state **MUST** be bit-identical across `HostSequential` and `HostThreaded` at 1, 2, 7 and 64 cores, and across native and `wasm32`, on the whole corpus.

Obligation 13.8 (Invariants). Requirement 9.3’s assertions hold on every step of every corpus run; Requirement 4.2 holds for every consumer shape; the counterexamples of Errata 9.1, 9.2 and 6.3 are regression tests with the corrected expectations.

Obligation 13.9 (Device conformance). A conformance corpus of `.cmat` files, seeds and expected step traces **MUST** be published from `f1r3comb-check` for use by external backends, and regenerated only by an explicit command.

Obligation 13.10 (The width of real programs). [1, Obl. 13.7], unchanged in content: measure the width distribution of compiled K0 programs with Requirement 11.2. It needs `F1R3COMB` through its milestone M2. It gates Section 12.4 and nothing else in this document.

14 Errors and diagnostics

Diagnostics follow [2, Req. 11.1]: a stable code, a message, no panic on any input, names rendered by content hash.

Code	Where	Meaning
<code>comb-mat-presentation</code>	any	Built or invoked under <code>presentation-b</code> (Decision 2.1)
<code>comb-feature-mismatch</code>	import	A <code>.cmat</code> or term with a different <code>shortcuts</code> setting; shared with <code>F1R3COMB</code>
<code>comb-name-budget</code>	step	Requirement 3.7; shared with <code>F1R3COMB</code>
<code>comb-mat-hash-collision</code>	boundary	Requirement 3.3
<code>comb-mat-overflow</code>	sparse, op	Checked <code>Nat</code> arithmetic overflowed
<code>comb-mat-decode</code>	import	Malformed <code>.cmat</code> : bad magic, version, tag, or an id referring forward
<code>comb-op-destructor</code>	op	Exploration of a term that can reach an <code>e</code> row, without a depth bound
<code>comb-op-budget</code>	op	Exploration budget exceeded
<code>comb-mat-internal</code>	any	An invariant assertion failed in a build that checks it; names the requirement

15 Performance, portability, determinism

Requirement 15.1 (Complexity). Import **MUST** be linear in the encoding size plus the interning of names. A step **MUST** cost $O(n \log n + E + F)$ for n rows, E enumerated redexes and F produced rows plus arena reads, with $n \log n$ replaceable by n under radix bucketing; it **MUST NOT** depend on the size of the intern table. Selection **MUST** be $O(E \log E)$ sequentially (the sort) and $O(\text{nnz}(B))$ beyond it.

Requirement 15.2 (Targets). Budget figures for a first implementation, on the reference machine of [5, §11], to be re-stated once measured: E1 at $k = 10^5$, $d = 8$ in under one second sequential; import of 10^5 atoms in under 100 ms; steady-state 10^7 fired redexes per second at 16 cores on wide terms.

Requirement 15.3 (Portability). As [2, Req. 12.3]: `wasm32-unknown-unknown` without `wasm-bindgen`, `forbid(unsafe_code)`, no external dependency. `HashMap` **MAY** be used only with a fixed-key hasher defined in the crate, never `RandomState`, because iteration order must never be observable and must never vary.

Requirement 15.4 (Determinism). Given the same term, seed, resolver and configuration, every trace, report and export MUST be bit-identical across runs, hosts, core counts, and WebAssembly versus native. No code path MAY iterate a hash map to produce an ordered result.

16 What the implementation asks of the papers

1. **Draft 3, Lemma 9.2 and Proposition 9.9** (Erratum 9.1). The lemma is false for $\mathbf{b}_l$, $\mathbf{b}_r$, $\mathbf{s}$ and the proposition’s bound fails on $\mathbf{b}_l(a, b) \mid \mathbf{m}(a, v) \mid \mathbf{m}(v, x)$. Proposed amendment: restate the lemma for the grading W , and the bound as $W(T) \leq 2N$. Strong normalisation, and the lattice edge it supports, are unaffected.
2. **Matrix note, Theorem 6.1(ii) and Corollary 6.2**. The nilpotency index is $W(T) + 1$, not $N + 1$, and the closure needs $\lceil \log_2(W + 1) \rceil$ squarings. The grading is the one in item 1.
3. **Matrix note, Theorem 6.1(iii)** (Erratum 9.2). Restore draft 3’s qualification: the non-increasing quantity is messages plus stores.
4. **Matrix note, Proposition 5.3 versus the reference** (Erratum 6.3). The proposition is right and the reference’s enumeration is not; say which count `run_sequential` samples from, since [1, Obl. 13.1] compares resolvers by their distributions.
5. **Matrix note, Remark 3.1 and F1R3COMB Decision 4.9**. State in both places that the `q` payload is sorted as a process and represented as its quotation, and that the shape tag carries the distinction the logic needs (Decision 2.2). As written, a reader of the two documents together sees a contradiction that is not there.
6. **F1R3COMB v0.2, Requirement 4.7**. Move the rule table to `f1r3comb-term` (Decision 2.4), and give it the fields F1R3COMB-MAT reads: premise subjects, datum kind, conclusion template, class.
7. **F1R3COMB v0.2, Requirement 8.3**. Scope the prohibition on decoding by projection to the CAMPF1R3 machine and to builds where `presentation-b` is reachable, citing Decision 2.3.
8. **Draft 3, Remarks 3.10 and 6.7**. State the rules of `conse` and `consfw` (Decision 2.5).
9. **Matrix note, Remark 8.7**. With shared priorities the greedy and the parallel independent sets coincide (Proposition 7.5). The remark can say so, and P4’s exit criterion then follows rather than having to be engineered.
10. **Matrix note, Obligation 13.2** is discharged by F1R3COMB’s encoding hash plus in-run structural interning (Requirement 3.2).
11. **Reference, experiments.py**. The E4 docstring says each gadget “forwards” the constructed name before opening it; the code builds `consm(a, b, c) \mid m(a, u) \mid m(b, v) \mid e(c)`, with no forwarder, and allocates an unused name `f`. The measurements are right for the code.

17 Open questions and deferred work

1. **Resolver fidelity and fairness** ([1, Obl. 13.1, Obl. 13.6]). F1R3COMB-MAT makes the resolver a named parameter and makes all three reproducible, which is what the comparison needs; the comparison itself is research.
2. **Contention remedies** (Requirement 6.9). Which join, and whether priority-ordered lazy enumeration can meet the identical-step requirement cheaply.

3. **Contention-set reconciliation** ([1, Obl. 13.4]). The hypergraph selection of Section 7 against the contention-set normalisation of the graded where-clause work. F1R3COMB-MAT’s selection is fixed by Definition 7.4 until that reconciliation says otherwise.
4. **Garbage** ([1, Obl. 13.3]). Inert rows are counted (Requirement 11.1); sidelining them from the joins is cheap and is a v0.2 candidate. Intern-table collection needs reachability from live rows and is not attempted.
5. **The history unfolding** ([1, §6.1, route (1)]), for triangularity with **e** present.
6. **Occupancy and nilpotency** ([1, Obl. 13.5]). With the operator layer in hand, whether the logic’s decidable-occupancy requirement coincides with Erratum 9.1’s grading is testable on the lattice corpus of [2, Req. 9.3].
7. **Weights**. A **Semiring** instance with per-rule-instance factors D_r [1, §12]; separate note.

18 Delivery

M	Deliverable	Done when
X0	f1r3comb-mat	Intern table, shape tables, import, export, .cmat , sparse layer; round trip (Obl. 13.1); key/hash agreement; canonical .cmat under $\equiv$. Needs F1R3COMB M0
X1	Finders	Rule table moved to f1r3comb-term ; join plans derived; Naive , Product , Bucketed ; symmetry mask; finder agreement (Obl. 13.2)
X2	f1r3comb-par , sequential	Priority, greedy step, commit order, budgets, reports, traces; Python parity (Obl. 13.3); trace certification (Obl. 13.4); invariants (Obl. 13.8)
X3	The width gate	comb-width ; Obligation 13.10 run on the F1R3COMB corpus and reported. Needs F1R3COMB M2. Gate on the device seam
X4	Threaded host	HostThreaded ; determinism across cores and wasm32 (Obl. 13.7); differential tests (Obl. 13.5); targets re-stated
X5	f1r3comb-op	Successors, exploration, T , closure, pre ; operator correctness (Obl. 13.6); comb-op
X6	Device conformance	Corpus and expected traces published (Obl. 13.9); external backend work only if X3 says so

X0–X2 are the representation and a correct machine, independently useful: they give [1] a checked port of its reference, and they give F1R3COMB a second, differently resolved machine to test against. X3 is the single number on which the accelerator enterprise stands or falls [1, Princ. 10.4]. X5 is independent of X3 and X4, and its natural consumer is the logic.

A Correspondence with the reference implementation

rhocombmat.py	F1R3COMB-MAT
ARITY, SHAPES	Shape, arities from the rule table; cons is <code>Shape::Cons (cons_i)</code>
NameTable.quote / drop	InternTable::quote / drop, keyed structurally (Def. 3.1)
State.occ (dict of occurrence ids)	Shape tables; occurrence identity is the row ref, stable within a step (Req. 8.1)
incidence	Implicit (Def. 3.10); <code>Csr::from_column</code> for the Product finder
redexes_matrix	Product finder, plus the symmetry mask
redexes_naive	Naive finder
BINARY_M, the quote join, the 3- and 4-premise joins	Derived join plans (Def. 4.4)
conflict_graph ($BB^\top$)	Reference only; production uses $B^\top$ (Req. 7.2)
mis (shuffled greedy)	Greedy by mix64 priority (Def. 7.4); identical to the rounds algorithm
fire	commit, with fixed effect order
run_parallel	<code>Resolver::MaximalProgress</code>
run_sequential	<code>Resolver::SingleUniform</code>
experiments.fan, contended, dup_tree, construct_then_eval	f1r3comb-check families E1–E4, golden against results.txt
validate	Obligation 13.2
q payload held as a name	Decision 2.2

References

- [1] L. G. Meredith. *Rho combinators as sparse linear algebra: a classical route from reflective concurrency to data-parallel execution*. Version 1, publications/rho-combinators/rhocomb-matrix, with rhocombmat.py, experiments.py and results.txt, September 2026.
- [2] F1R3FLY.io. *F1R3Comb: a compiler from kernel rho to the name-free rho combinators, and a machine that runs the result*. Specification v0.2, publications/rho-combinators/f1r3comb-spec-v02, September 2026.
- [3] L. G. Meredith and M. Stay. *Name-free combinators for the rho calculus: a repair, a linear encoding, and the one operation that was missing*. Draft 3, publications/rho-combinators, September 2026.
- [4] F1R3FLY.io. *A spatial-behavioural logic for the rho combinators*. Draft 2, publications/rho-combinators/rhocomb-logic, September 2026.
- [5] F1R3FLY.io. *K1ndllng: a parser and normaliser for kernel rholang*. publications/campf1r3/k1ndllng-spec, 2026.
- [6] F1R3FLY.io. *CampF1R3: an in-memory tuple space*. publications/campf1r3/campf1r3-spec, 2026.
- [7] G. E. Blelloch, J. T. Fineman and J. Shun. Greedy sequential maximal independent set and matching are parallel on average. *SPAA*, 2012.

- [8] M. Luby. A simple parallel algorithm for the maximal independent set problem. *SIAM Journal on Computing*, 1986.
- [9] F.G. Gustavson. Two fast algorithms for sparse matrices: multiplication and permuted transposition. *ACM Transactions on Mathematical Software*, 1978.
- [10] G.L. Steele, D. Lea and C.H. Flood. Fast splittable pseudorandom number generators. *OOPSLA*, 2014.
- [11] H. Q. Ngo, E. Porat, C. Ré and A. Rudra. Worst-case optimal join algorithms. *Journal of the ACM*, 2018.
- [12] J. Kepner and J. Gilbert (eds.). *Graph Algorithms in the Language of Linear Algebra*. SIAM, 2011.
- [13] J. Baez and J. Biamonte. *Quantum Techniques for Stochastic Mechanics*. World Scientific, 2018.